

Marco Betti, Minh Phan, and Isaac Li

CSC 246

## Final Project

### Introduction/Data Summary

The main objective of this project is to build a CNN from scratch and evaluate its performance on the MNIST and Fashion MNIST datasets. We also implemented a CNN using PyTorch libraries which ended up being much faster than our DIY CNN. The DIY CNN consists of a convolutional layer, followed by ReLU activation, max pooling, flattening, and two fully connected layers with ReLU activation and softmax output. The PyTorch CNN is designed to have a similar architecture to the DIY CNN for comparison purposes.

Both models are trained and evaluated on the MNIST dataset, which consists of 70,000 grayscale images of handwritten digits (0-9), with 60,000 images used for training and 10,000 images for testing. Similarly, the Fashion MNIST dataset contains 70,000 grayscale images of fashion products from 10 categories, following the same split as MNIST. The dataset is split into training and validation sets to assess the models' performance.

The DIYCNN implementation can be found in DIYCNN.ipynb. And the PyTorch implementation can be found in torch.ipynb.

NOTE: The DIY CNN is trained using local csv files which were too big to upload to gradescope. These CSV files are downloaded from the following links:

MNIST: <https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

Fashion-MNIST: <https://www.kaggle.com/datasets/zalando-research/fashionmnist>

To run the DIY model, download these files and place them in the same folder. More specifically, the files required are "mnist\_train.csv", "mnist\_test.csv", "fashion-mnist\_train.csv", and "fashion-mnist\_trest.csv".

The PyTorch CNN on the other hand has code that automatically downloads the data using torch.

### Methods

The CNN architecture used in this project consists of the following components:

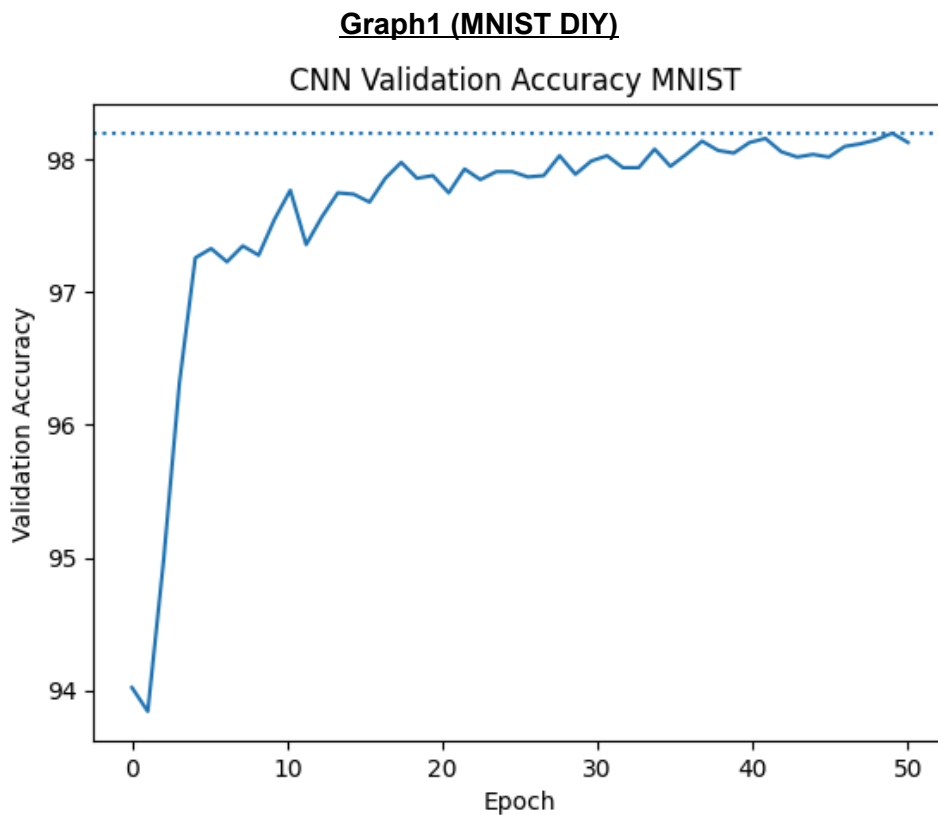
- Convolutional Layers:
  - 1 convolutional layer with 8 filters
  - The convolutional layer uses a kernel size of 3

- ReLU activation function is applied after the convolutional layer
- Max pooling layer with a kernel size of 2 is applied after the ReLU activation
- Fully Connected Layers:
  - Flatten layer to convert the 2D feature maps to a 1D vector
  - First linear layer with 1352 input units and 100 output units
  - ReLU activation function is applied after the first linear layer
  - Second linear layer with 100 input units and 10 output units
  - Softmax activation function is applied to obtain class probabilities

The CNN is trained using the Stochastic Gradient Descent (SGD) optimizer with a learning rate of 0.01. The batch size is set to 100, and the model is trained for 50 epochs. The loss function used is the Negative Log-Likelihood (NLL) loss.

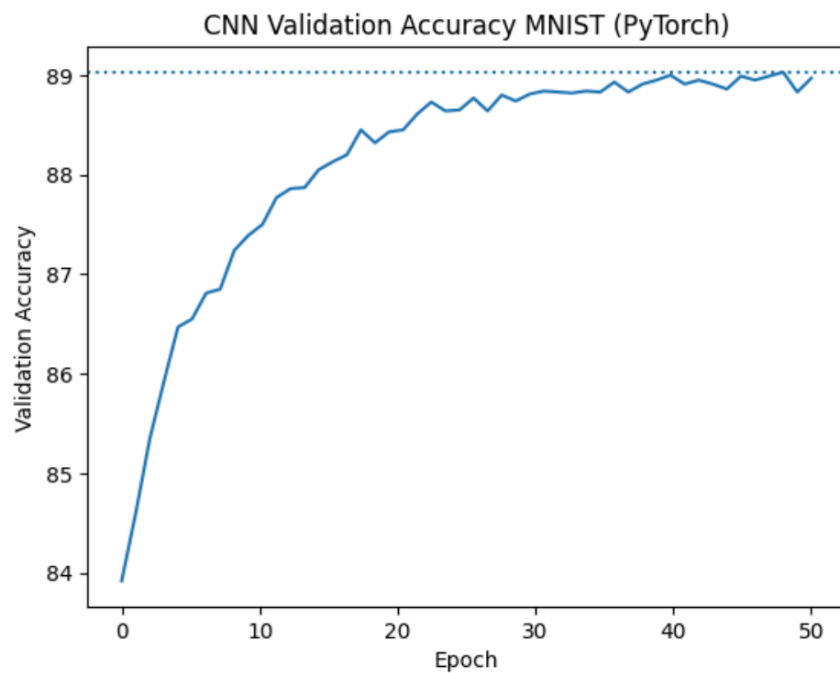
## **Experiments and Results**

The performance of the custom CNN implementation is evaluated on both the MNIST and Fashion MNIST datasets. The model is trained over 50 epochs and the validation accuracy is recorded at the end of each epoch. The training progress is visualized using plots of the validation accuracy over the course of training. The same process is carried out with the PyTorch CNN for comparison.



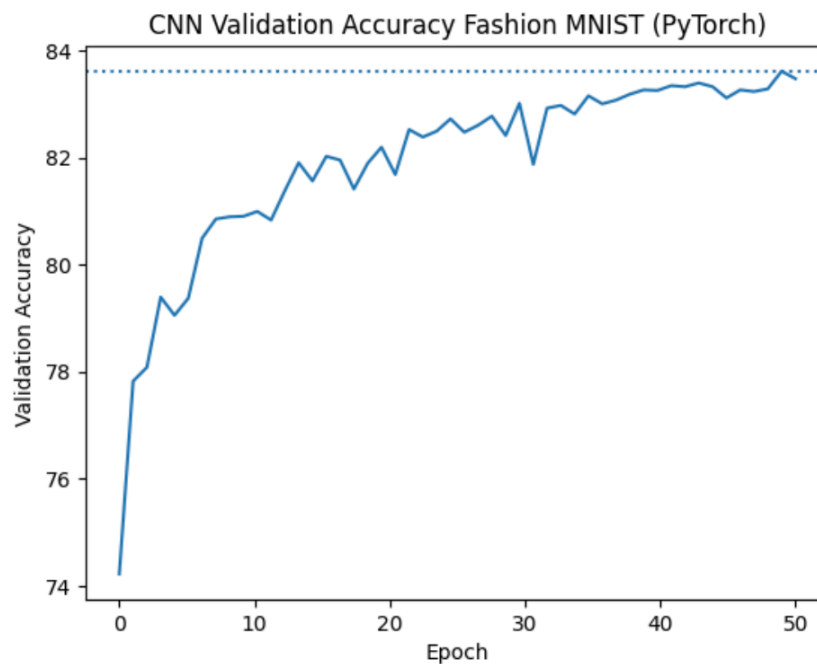
On the MNIST dataset, the custom CNN achieves a maximum validation accuracy of 98.2 after 50 epochs of training. This entire process took over 400 minutes to complete.

**Graph 2 (PyTorch MNIST results)**



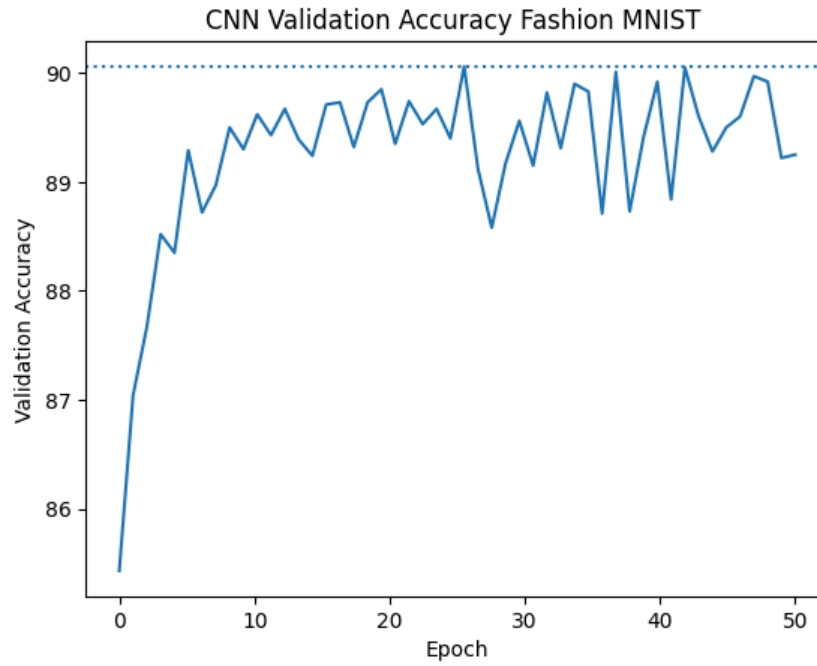
The PyTorch made CNN achieves a maximum accuracy of 89.03% after 50 epochs on the MNIST dataset

**Graph 3 (Fashion-MNIST PyTorch)**



The PyTorch made CNN achieves a maximum accuracy of 83.61 after 50 epochs on the Fashion MNIST dataset. This was actually out performed by our DIY CNN which achieved a whopping 90.1% accuracy on the Fashion MNIST dataset.

**Graph 4 (Fashion-MNIST DIY)**



Similarly, on the Fashion MNIST dataset, the custom CNN reaches a maximum validation accuracy of 90.1 after 50 epochs. The model's performance on Fashion MNIST is slightly lower compared to MNIST, which can be attributed to the higher complexity and variability of the fashion product images. Similarly to when our DIY CNN was trained on the MNIST dataset, it took over 400 minutes to complete. Totalling for about 800-850 minutes time to complete training both. This was done on a Macbook pro with an M1 max chip.

It is important to note that the data was not processed in the same way between the PyTorch CNN and our DIY CNN. We tried to keep them consistent, but ended up having to use Torch for the PyTorch implementation, and Pandas for our DIY CNN. However this could lead to disparity in results. Apart from this, the implementations CNN that we made for this project use the same parameters, such as a learning rate of 0.01

It is also worth noting that the custom CNN implementation takes approximately 10 + times longer to train compared to the equivalent PyTorch built-in functions. The first version of our DIY CNN was even slower, and took about 1 hour to complete 1 epoch. We then decided to optimize this by making the operations in the Conv2d class more efficient. We did this by using matrix multiplication instead of nested loops. This way we could take advantage of NumPy's highly optimized linear algebra routines, which significantly sped up the computations. After this update our CNN could now run 1 epoch in about 10 minutes, speeding it up by almost 6 fold.

## **Conclusion/Discussion**

The Pytorch Implemented CNN beat ours in efficiency, but even so our DIY CNN had very successful results, which demonstrated its ability to learn. Comparing the performance between the two datasets, it is evident that both models achieve higher accuracy on MNIST compared to Fashion MNIST. This can be attributed to the higher complexity and variability of the Fashion MNIST dataset, which contains images of different fashion products with more intricate patterns and details. Nevertheless, the custom CNN implementation demonstrates its ability to learn meaningful representations and make accurate predictions on both datasets. Since the dataset has 10 different classes, we can consider guessing to be about 10% accurate. And therefore accuracy rates higher, like the DIY CNN on fashion MNIST which got 90.1, demonstrates that our model was able to learn, and it learned very well. At the cost of computations taking multiple hours to train.

In the future, Further exploration could involve investigating the impact of different architectural choices, such as varying the number and size of convolutional layers, or experimenting with alternative activation functions. Additionally, the current implementation could probably be optimized to reduce the training time and improve computational efficiency.